Karunya INSTITUTE OF TECHNOLOGY AND SCIENCES
(Declared as Deemed to be University under Sec.3 of the UGC Act, 1956)
MoE, UGC & AICTE Approved
NAAC A++ Accredited

Division of Electronics and Communication Engineering 2026-2027 (ODD SEM)

III IA EVALUATION REPORT

for

COMPUTER NETWORKS – PROJECT BASED COURSE

Title of the project: "Design and Implementation of a Simple File Transfer Application Using TCP Socket Programming"

A report submitted by

Name of the Student	***SHIVABALAN K.B*** ***PRANOV RAGHAVENDHRA.MD***
Register Number	***URK23EC1055*** ***UTK23EC1001***
Subject Name	***COMPUTER NETWORKS***
Subject Code	***18EC2017***
Date of Report submission	***30/10/2026***

Project Rubrics for Evaluation

First Review: Project title selection – PPT should have four slides (Title page, Introduction, Circuit/Block Diagram, and Description of Project).

Second Review: PPT should have three slides (Description of Concept, implementation, outputs, results and discussion).

Total marks: / 40

Marks Signature of Faculty with date:

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO.
1.	ABSTRACT	3
2.	INTRODUCTION	4
3.	DESCRIPTION OF THE PROJECT	6
4.	CONCEPT INVOLVED	8
5.	SYSTEM PARAMETERS AND PROTOCOL DATA	10
6.	TOOLS USED	12
7.	IMPLEMENTATION	13
8.	RESULTS WITH DISCUSSIONS	16
9.	INFERENCES	18
10.	CONCLUSION	19
11.	REFERENCES	20

CHAPTER 1
ABSTRACT

This project presents the design and implementation of a simple peer-to-peer file transfer application that uses Python TCP socket programming to move files between two computers on the same local network. The application is accessed through a browser-based interface served by a lightweight FastAPI web server, so the user does not need any specialised client software.

The transfer is carried out with an application-level protocol built on top of TCP. The sender first transmits file metadata (file name, file size and a SHA-256 checksum), the receiver validates it and replies with an acknowledgement, the file is then streamed in fixed 4096-byte chunks, and finally the receiver confirms success or failure after verifying the size and the checksum.

Important networking concepts are demonstrated in a working system: the TCP three-way handshake, connection-oriented communication, length-prefix message framing, handling of partial reads, acknowledgements and integrity verification. The receiver also protects itself against unsafe file names and never overwrites an existing file.

The application was tested on localhost with text files and binary files of 1 KB, 500 KB and 5 MB, and with error cases such as a receiver that is not running and duplicate file names. All tests passed and the received files matched the originals bit for bit.

CHAPTER 2
INTRODUCTION

Computer networks allow independent machines to exchange data. When two users on the same Wi-Fi network want to share a file, the common solutions are cloud storage, e-mail or removable drives. Cloud services need internet access and a third-party server, e-mail limits the attachment size, and a pen drive needs physical contact. A direct transfer over the local network avoids all of these problems.

The Transmission Control Protocol (TCP) works at the transport layer of the OSI model and provides reliable, ordered and error-checked delivery of a byte stream. These properties make TCP the natural choice for file transfer, because even one missing or corrupted byte can make a file unusable.

A socket is the programming interface through which an application uses TCP. A socket is identified by an IP address and a port number. In this project the Python standard library module socket is used to create a listening server on the receiving computer and a client on the sending computer.

Since a web browser cannot open raw TCP sockets, a FastAPI backend is placed between the browser and the network. The browser talks to the local backend over HTTP, and the backend talks to the remote receiver over TCP. This gives the user a simple graphical interface while the core data transfer still uses real socket programming.

2.1 Background

Data communication between computers is organised into layers. The application layer holds programs such as web browsers and file transfer tools, the transport layer (TCP or UDP) provides end-to-end delivery between processes, the network layer (IP) moves packets between networks, and the lower layers send bits over the physical medium. A file transfer program normally sits at the application layer and relies on the transport layer for delivery.

Layer	Protocol / Item	Role in this project
Application	Custom file transfer protocol	Metadata, ACK, chunks, SUCCESS / FAILURE
Transport	TCP	Reliable, ordered byte stream
Network	IP (IPv4 address)	Addresses the receiving computer
Link / Physical	Wi-Fi or Ethernet	Carries frames on the local network

Table 2.1 Layers involved in the file transfer

2.2 Motivation

Students, teams and small offices often need to move large files between two machines on the same network. Cloud storage consumes internet bandwidth and uploads private data to a third

party, e-mail rejects large attachments, and removable drives need physical access. A direct transfer over the local network is faster, private and independent of the internet.

Building such a tool is also an effective way to understand how real networked software works. Writing the sender and receiver from the socket level forces the designer to deal with the practical issues that textbooks describe: connection set-up, message boundaries, partial reads, timeouts, error recovery and data verification.

2.3 Why TCP and Not UDP

Feature	TCP	UDP
Connection	Connection-oriented	Connectionless
Delivery	Guaranteed, retransmitted	Not guaranteed
Ordering	Preserved	Not preserved
Speed	Slightly slower	Faster
Suitable for	File transfer, web, e-mail	Streaming, games, DNS

Table 2.2 Comparison of TCP and UDP

A file is useful only if every byte is correct. With UDP a lost datagram would silently damage the file and the application would have to build its own retransmission and ordering. TCP already provides these services, so it is the correct choice for this project.

2.4 Scope of the Project

The project covers transfer of a single file at a time between two computers on the same local area network. It includes the sender, the receiver, the application protocol, the browser interface and the test procedure. Encryption, authentication and transfer across the internet through routers and firewalls are outside the scope and are listed as future work.

CHAPTER 3
DESCRIPTION OF THE PROJECT

3.1 Problem Statement

To transfer a file directly from one computer to another on the same local network, without using cloud services, and to confirm that the file arrived completely and correctly.

3.2 Objectives

- To implement TCP socket based file transfer using the Python standard socket library.
- To design a simple application-level protocol with metadata, acknowledgements and checksums.
- To provide a browser-based interface for sending and receiving files.
- To handle errors such as connection failure, corrupted data and duplicate file names.
- To verify file integrity using SHA-256.

3.3 Features

- Send a file by choosing it in the browser and entering the receiver's IP address and port.
- Receive files by starting a TCP listener with a chosen destination folder.
- Real-time progress bar on both the sending and the receiving side.
- SHA-256 integrity check for every received file.
- Duplicate protection: an existing file is never overwritten; a numbered suffix is added.
- Persistent transfer history with file name, size, direction, time and status.

3.4 System Architecture

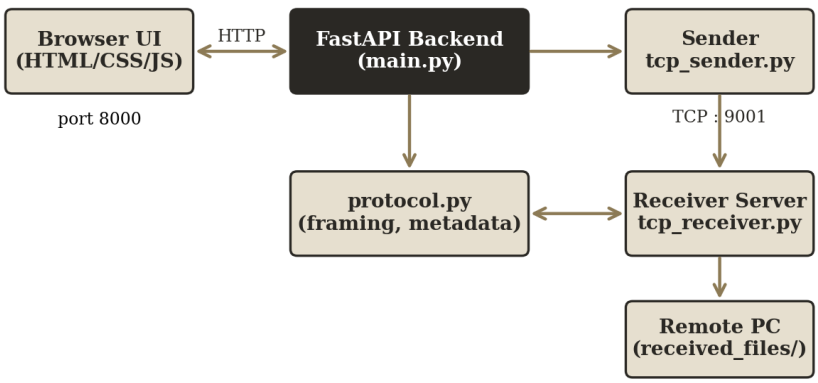


Fig. 3.1 System architecture of the file transfer application

The browser sends the selected file to the FastAPI backend over HTTP (port 8000). The backend calls the TCP sender, which connects to the receiver's TCP server (default port 9001). On the other computer, the receiver server runs in a background thread, saves the file and reports events that the browser reads by polling.

3.5 Requirements

The functional and non-functional requirements of the application are summarised below.

Type	Requirement
Functional	Select a file, enter the receiver IP address and port, and send it
Functional	Start and stop a receiver on a chosen port and folder
Functional	Show progress and a final success or failure message
Functional	Keep a history of sent and received files
Non-functional	Reliability: the received file must be identical to the original
Non-functional	Safety: unsafe file names rejected, no overwriting
Non-functional	Usability: operated entirely from a web browser
Non-functional	Robustness: timeouts and clear error messages

Table 3.1 Requirements

3.6 Working Procedure

The complete operation can be understood as two cooperating procedures, one on each computer.

Receiver side: The user opens the web page, selects a destination folder and a listening port (default 9001) and clicks Start Receiving. The backend creates a TCP server socket, binds it to all interfaces, and starts a background thread that waits for connections. The page shows that the receiver is listening.

Sender side: The user opens the web page on the other computer, chooses a file, types the receiver's IP address and port and clicks Send File. The browser uploads the file to the local backend, which computes the SHA-256 checksum, connects to the receiver, and runs the transfer protocol described in Chapter 5 and Chapter 7.

Completion: When the receiver has verified the size and checksum it sends SUCCESS. Both computers show a success message and record the transfer in the history table. If anything goes wrong, a failure message with the reason is shown and the partial file is removed.

3.7 User Interface

The interface is a single web page with three areas. The Receive a File area holds the destination folder, the port number and the Start Receiving and Stop buttons together with a status indicator. The Send a File area holds the file picker, the receiver IP address, the port and the Send File button with a progress bar. The Transfer History table lists every transfer with its direction, file name, size, time and status. A separate How It Works page explains the protocol for learners.

CHAPTER 4
CONCEPT INVOLVED

4.1 Transmission Control Protocol

TCP is a connection-oriented transport protocol. Its main properties are listed below.

Property	Meaning
Reliable delivery	Lost packets are retransmitted automatically
Ordered delivery	Data arrives in the same order in which it was sent
Error detection	Checksums detect corrupted segments
Flow control	A fast sender cannot overwhelm a slow receiver
Congestion control	Sending rate is reduced when the network is congested

Table 4.1 Properties of TCP

4.2 Three-Way Handshake

Before any data is sent, TCP establishes a connection using three messages: the client sends SYN, the server replies with SYN-ACK, and the client answers with ACK. In this project the operating system performs the handshake when the sender calls connect().

4.3 Socket Programming

The receiver creates a socket, binds it to an address and port, listens and accepts connections. The sender creates a socket and connects to the receiver's address. Data is sent with sendall() and read with recv().

4.4 Partial Reads and Message Framing

TCP is a byte stream and has no message boundaries. A single recv() call may return fewer bytes than requested. To solve this, the project uses a helper function recv_exact() that keeps reading until exactly the required number of bytes has arrived, and length-prefix framing in which every control message is preceded by a 4-byte big-endian length field.

4.5 SHA-256 Integrity Check

SHA-256 produces a 256-bit (64 hexadecimal character) digest of the data. Any change in the input, even a single bit, gives a completely different digest. The sender computes the digest before sending and the receiver recomputes it after receiving; if the two values differ, the file is reported as corrupted.

4.6 Client–Server Model

The two programs play different roles. The receiver is the server: it opens a socket, waits for a connection and serves each incoming transfer. The sender is the client: it initiates the connection to a known address and port. Since the same application is installed on both computers, any computer can act as sender or receiver, which gives the project its peer-to-peer character.

4.7 IP Address and Port Number

An IP address identifies a computer on the network, for example 192.168.1.5, and a port number identifies one application on that computer. The pair (IP address, port) forms a socket address. The receiver in this project listens on port 9001 and the web interface uses port 8000. Port numbers from 1024 to 65535 are normally free for user programs. On Windows the IP address of the receiver can be found with the command ipconfig.

4.8 Why the File is Sent in Chunks

Reading a complete file into memory before sending it would fail for large files. The sender therefore reads the file in blocks of 4096 bytes and sends each block in turn. The receiver reads the same amount, writes it to disk and updates the running hash. In this way memory use stays small and constant, whatever the size of the file, and progress can be reported after every block.

4.9 Multithreading

A server that handled a transfer inside its accept loop could not respond to anything else during that time. The receiver therefore runs the accept loop in a background thread and creates a new thread for each connection. The web server stays responsive, so the browser can keep polling the progress while the data is arriving.

4.10 Timeouts and Error Recovery

Network operations can block forever if the other side disappears. Socket timeouts are used so that a stalled connection is detected and reported. If the connection ends before all the bytes of the file have arrived, the receiver removes the incomplete file and sends FAILURE, so an unusable half file is never left on the disk.

CHAPTER 5
SYSTEM PARAMETERS AND PROTOCOL DATA

5.1 Configuration Parameters

Parameter	Value	Purpose
Chunk size	4096 bytes	Size of each file block
Frame header	4 bytes	Big-endian length prefix
Transfer port (default)	9001	TCP file transfer
Web server port	8000	HTTP for the browser
Timeouts	30 s / 60 s	Control messages / file data
Hash function	SHA-256	Integrity verification

Table 5.1 System parameters

5.2 Control Messages

Message	Sent by	Meaning
Metadata (JSON)	Sender	File name, file size and checksum
ACK	Receiver	Metadata is valid, send the file
REJECT	Receiver	Metadata or file name is invalid
SUCCESS	Receiver	Size and checksum matched
FAILURE	Receiver	Size or checksum mismatch

Table 5.2 Control messages of the protocol

5.3 Frame Format

Every control message is sent as a 4-byte unsigned integer in big-endian order that gives the payload length N, followed by N bytes of payload (JSON or an ASCII token). File data is not framed; it is streamed directly, and the receiver stops after the number of bytes declared in the metadata.

5.4 Metadata Message

The first message from the sender is a JSON object that describes the file. An example is shown below.

```
{
  "filename": "report.pdf",
  "filesize": 524288,
  "checksum": "eed9f1037af064e3...a8b9fb4d"
}
```

The receiver reads these three fields, validates the file name, and decides whether to accept the transfer. The file size tells it how many bytes of file data will follow, and the checksum is kept for the final comparison.

5.5 Receiver Decisions and Responses

Condition	Reply	Action on receiver
Metadata unreadable or incomplete	REJECT	Report error, close
File name unsafe	REJECT	Report error, close
File name already exists	ACK	Save under a new numbered name
Connection lost during data	FAILURE	Delete partial file
Size does not match	FAILURE	Delete file
Checksum does not match	FAILURE	Delete file
Everything matches	SUCCESS	Keep file, log event

Table 5.3 Receiver decisions

5.6 File Name Rules

A received file name is first reduced to its base name, which removes any directory part, and is then compared with a pattern that allows only letters, digits, hyphens, dots, spaces and underscores. A name such as ../../secret.txt is therefore refused. If the name is acceptable but a file with that name already exists in the destination folder, the receiver tries name_1.ext, name_2.ext and so on until it finds a free name.

CHAPTER 6
TOOLS USED

Layer	Technology
Backend	Python 3, socket (standard library), FastAPI, Uvicorn
Frontend	HTML5, CSS3, Vanilla JavaScript
File integrity	hashlib (SHA-256)
File upload to backend	python-multipart
Browser	Google Chrome

Table 6.1 Tools and technologies

Python was chosen because its socket module gives direct access to TCP with very little code. FastAPI provides a fast and simple way to expose the sender and receiver functions as HTTP endpoints, and Uvicorn is used as the server.

6.1 Project Structure

File	Role
backend/main.py	FastAPI application and HTTP endpoints
backend/tcp_sender.py	TCP file sender logic
backend/tcp_receiver.py	TCP receiver server and per-connection handler
backend/protocol.py	Framing, metadata encoding and decoding
frontend/index.html, script.js, style.css	Browser interface and logic
frontend/how-it-works.html	Explanation page for the protocol

Table 6.2 Files of the project

6.2 Software Used

Tool	Purpose
Python 3.10 or newer	Programming language for the backend and the protocol
socket module	Creating TCP client and server sockets
hashlib module	Computing SHA-256 digests
struct and json modules	Frame headers and metadata encoding
threading module	Background receiver and per-connection threads
FastAPI	HTTP endpoints for the browser
Uvicorn	Server that runs the FastAPI application
Google Chrome	Browser used for the interface and demonstration

Table 6.3 Software tools

CHAPTER 7
IMPLEMENTATION

7.1 Transfer Sequence

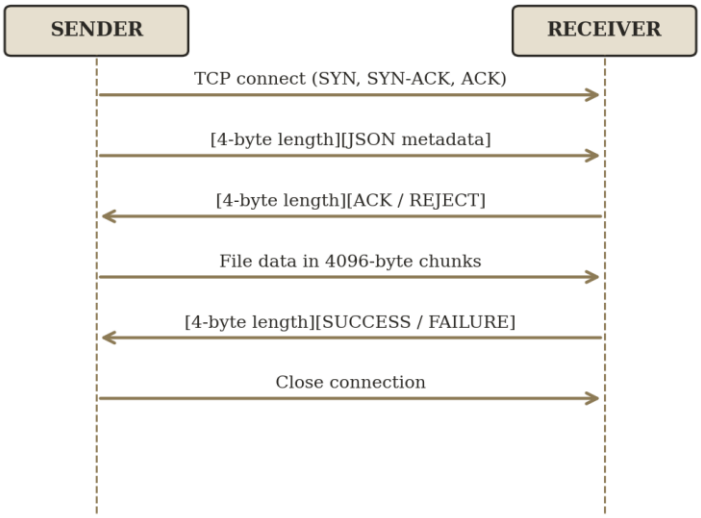


Fig. 7.1 File transfer sequence between sender and receiver

7.2 Reliable Reading (protocol.py)

```
def recv_exact(sock, n):
    buf = b""
    while len(buf) < n:
        chunk = sock.recv(n - len(buf))
        if not chunk:
            raise ConnectionError("Connection closed")
        buf += chunk
    return buf
```

The function loops until exactly n bytes are collected and raises an error if the connection closes early, so the partial read problem is handled.

7.3 Sender (tcp_sender.py)

The function send_file() computes the SHA-256 checksum, connects to the receiver, sends the framed metadata and waits for ACK or REJECT. After an ACK it reads the file in 4096-byte chunks and sends each chunk with sendall(), reporting progress through a callback. Finally it waits for SUCCESS or FAILURE and returns a result dictionary to the backend.

7.4 Receiver (tcp_receiver.py)

The ReceiverServer class runs a listening socket in a background thread and starts a new thread for each connection. For every connection the handler reads the metadata, sanitises the file name using os.path.basename() and a regular expression, chooses a unique file name, sends ACK, writes the incoming chunks to disk while updating the hash, compares the size and checksum, and sends SUCCESS or FAILURE. If the connection drops, the incomplete file is deleted.

7.5 Backend Endpoints (main.py)

Endpoint	Method	Function
/send	POST	Receive the file from the browser and call send_file()
/receiver/start	POST	Start the receiver server on a port
/receiver/stop	POST	Stop the receiver server
/receiver/status	GET	Return pending events, polled every 800 ms
/history	GET	Return the transfer history

Table 7.1 HTTP endpoints

7.6 Procedure to Run

Install the packages with pip install -r requirements.txt, then start the server with python -m uvicorn backend.main:app --host 0.0.0.0 --port 8000 and open http://localhost:8000 in Chrome. On the receiving computer click Start Receiving; on the sending computer choose a file, enter the receiver's IP address and port, and click Send File.

7.7 Message Framing Functions (protocol.py)

```
def send_framed(sock, payload: bytes) -> None:
    header = struct.pack(">I", len(payload))
    sock.sendall(header + payload)

def recv_framed(sock) -> bytes:
    header = recv_exact(sock, 4)
    payload_len = struct.unpack(">I", header)[0]
    return recv_exact(sock, payload_len)
```

send_framed() prepends the 4-byte length and sends the message in one call. recv_framed() first reads the 4-byte header, decodes the length and then reads exactly that many bytes, so message boundaries are always preserved.

7.8 Sending the File in Chunks (tcp_sender.py)

```
sock.connect((receiver_ip, receiver_port))
send_framed(sock, build_metadata(filename, filesize, checksum))
response = recv_framed(sock)       # ACK or REJECT
...
with open(filepath, "rb") as f:
    while True:
        chunk = f.read(CHUNK_SIZE)
        if not chunk:
            break
        sock.sendall(chunk)
        bytes_sent += len(chunk)
final = recv_framed(sock)          # SUCCESS or FAILURE
```

After the receiver accepts the metadata, the file is read block by block and each block is sent with sendall(), which keeps sending until the whole block has been handed to TCP. The final reply decides whether the transfer is reported as successful.

7.9 Filename Safety (tcp_receiver.py)

```
SAFE_FILENAME_RE = re.compile(r'^[\w\-. ]+$')

def _sanitize_filename(name):
    base = os.path.basename(name).strip()
    if not base or not SAFE_FILENAME_RE.match(base):
        return None
    return base
```

7.10 Receiving and Verifying the File

```
h = hashlib.sha256()
with open(save_path, "wb") as f:
    while bytes_received < filesize:
        to_read = min(CHUNK_SIZE, filesize - bytes_received)
        chunk = recv_exact(conn, to_read)
        f.write(chunk)
        h.update(chunk)
        bytes_received += len(chunk)

if h.hexdigest() != checksum:
    send_framed(conn, FAILURE)
    os.remove(save_path)
else:
    send_framed(conn, SUCCESS)
```

7.11 Starting the Receiver Server

```
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
sock.bind(("0.0.0.0", port))
sock.listen(5)
sock.settimeout(1.0)     # re-check the stop flag every second

while not stop_event.is_set():
    try:
        conn, addr = sock.accept()
    except socket.timeout:
        continue
    threading.Thread(target=_handle_connection,
                     args=(conn, addr, dest, on_event)).start()
```

Binding to 0.0.0.0 accepts connections on every interface; the one-second timeout lets the thread stop cleanly.

CHAPTER 8
RESULTS WITH DISCUSSIONS

The application was tested on a Windows machine using localhost. The results are given below.

No.	Test	Observation	Result
1	Small text file (1 KB)	Received correctly	PASS
2	Binary file (500 KB) with SHA-256	Checksums matched	PASS
3	Receiver not running	Clear connection error shown	PASS
4	Duplicate file name	file.txt saved as file_1.txt	PASS
5	Large file (5 MB) with SHA-256	Checksums matched	PASS

Table 8.1 Test results

8.1 Discussion

- Integrity: for the binary and large files the checksum computed by the sender was identical to the one computed by the receiver, which shows that no data was lost or altered.
- Error handling: when no receiver was listening, the sender returned a readable ‘Cannot connect’ message instead of failing silently.
- Safety: existing files were never overwritten, because a numbered suffix is added to a duplicate name.
- Feedback: the progress bar and the transfer history table update during and after each transfer.

8.2 Limitations

- No encryption: data is sent in plain text, so the application is suitable only for a trusted LAN.
- No authentication: any machine on the network can connect to an open receiver.
- Only one active transfer at a time per application instance.
- The browser-to-backend step uses HTTP, because a browser cannot open raw TCP sockets.

8.3 Test Procedure

Each test was carried out by starting the receiver on port 9001, sending a file to 127.0.0.1, and then comparing the file in the received_files folder with the original. For the binary and large files the SHA-256 value of the original and of the received copy were compared. The transfer history was also checked after every test to confirm that the entry had been recorded with the correct status.

No.	Test case	Expected behaviour	Verification
1	1 KB text file	File saved unchanged	Content compared

No.	Test case	Expected behaviour	Verification
2	500 KB binary file	File saved, hashes equal	SHA-256 compared
3	Receiver stopped	Connection error shown	Message on screen
4	Same file sent twice	Second copy renamed with _1	Folder listing
5	5 MB file	File saved, hashes equal	SHA-256 compared

Table 8.2 Test cases and verification

8.4 Analysis of the Results

The first, second and fifth tests confirm the normal path of the protocol: connection, metadata, ACK, chunked transfer, verification and SUCCESS. The 5 MB file requires over a thousand chunks, and the matching hash shows that chunking and reassembly work correctly for files much larger than one block.

The third test shows the behaviour when the connection cannot be made. Because connect() fails immediately, the sender returns a clear message that names the address and port, so the user can see that the receiver has to be started or that the address is wrong.

The fourth test confirms the protection against overwriting. The first file kept its name and the second was saved as dup_test_file_1.txt, which is also visible in the received_files folder of the project.

CHAPTER 9
INFERENCES

- TCP gives reliable and ordered delivery, so a simple file transfer needs no retransmission logic in the application.
- Because TCP is a byte stream, an application protocol with length-prefix framing is needed to separate control messages.
- A recv() call can return less data than requested, so reading must always be done in a loop (recv_exact).
- A checksum gives an end-to-end check of the file that is independent of TCP's own error detection.
- Validating the file name and avoiding overwrites are essential for the safety of a receiving server.

9.1 Observations from the Implementation

While building and testing the application the following points were observed. They are the practical lessons of the project and show how theory in computer networks appears in real software.

- The protocol worked without any application-level retransmission, which shows how much work TCP does for the programmer.
- Control messages and file data use different methods in the same connection: control messages are framed with a length, while the file data is delimited by the size announced in the metadata.
- Timeouts are necessary, because without them a stalled connection would block the sender or receiver indefinitely.
- Running the receiver in its own thread keeps the interface responsive during long transfers.
- The application is limited by the speed of the local network and the disk, not by the protocol design.

9.2 Advantages

- Direct transfer without cloud storage, e-mail or external drives.
- No internet connection is required; only a shared local network.
- Integrity of every file is proved by SHA-256.
- Simple browser interface with progress and history.
- Small code base that uses only standard Python modules for the networking part.

9.3 Applications

- Sharing project files between computers in a laboratory or hostel network.
- Moving large files inside a small office without using cloud services.

- Teaching aid for socket programming and protocol design in a computer networks course.

CHAPTER 10
CONCLUSION

A working file transfer application was designed and implemented using Python TCP sockets, a custom application-level protocol and a FastAPI browser interface. The system transfers files between two computers on the same network, reports progress, verifies integrity with SHA-256, and handles connection failures and duplicate file names correctly. All planned tests were successful.

The project demonstrates, in a practical form, how connection-oriented communication, message framing and checksums work together to provide dependable data transfer.

Future Work

- TLS encryption using Python's ssl module.
- Password protection for the receiver.
- Multiple simultaneous transfers.
- Pause and resume support.
- Folder transfer by compressing it before sending.

Summary of Work Done

- Studied TCP, the three-way handshake, sockets, message framing and checksums.
- Designed an application-level protocol with metadata, ACK or REJECT, chunked data and SUCCESS or FAILURE.
- Implemented the sender, receiver and protocol modules in Python.
- Built a FastAPI backend and a browser interface with progress and history.
- Tested the system with text, binary and large files and with error situations.

The objectives set at the start of the project were therefore achieved. The work also shows that reliable network applications need careful treatment of details such as message boundaries, partial reads, timeouts and verification, even when a reliable transport protocol is available.

CHAPTER 11
REFERENCES

[1] Python Software Foundation, "socket – Low-level networking interface," Python 3 Documentation, https://docs.python.org/3/library/socket.html.

[2] Python Software Foundation, "hashlib – Secure hashes and message digests," Python 3 Documentation, https://docs.python.org/3/library/hashlib.html.

[3] FastAPI Documentation, https://fastapi.tiangolo.com.

[4] J. Postel, "Transmission Control Protocol," RFC 793, IETF, 1981.

[5] A. S. Tanenbaum and D. J. Wetherall, Computer Networks, 5th ed., Pearson, 2011.

[6] J. F. Kurose and K. W. Ross, Computer Networking: A Top-Down Approach, 8th ed., Pearson, 2020.

[7] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," FIPS PUB 180-4, 2015.